

Modelos de IA generativa en tu propia infraestructura

**IA: Actividades para todas las edades y
materias**

Madrid, España, 11 de mayo de 2026

Jesus M. Gonzalez-Barahona

<https://jgbarah.github.io/presentations/>

Traducido del original en inglés,
usando el modelo

`nvidia/nemotron-3-nano-30b-a3b`

¿Qué contiene un modelo generativo

- Arquitectura
- Pesos, solo pesos
- Software para inferir

Pero también:

- Datos para entrenar, comparar
- Software para entrenar, comparar
- Pesos de modelos intermedios
- Documentación, explicando todo

Un amplio espectro

- Modelo detrás de la aplicación
- Modelo directamente accesible
- Modelo con pesos disponibles
- Modelo de peso abierto
- Modelo de código abierto
- Modelo reproducible (libre)

Behind-app model

Directly accessible model

Available weights model

Open weight model

Open source model

Reproducible (libre) model

Clases de «apertura»

- Modelo abierto: arquitectura, parámetros del modelo, pesos y metadatos, informe técnico, tarjeta del modelo, tarjeta de datos
- Herramientas abiertas: modelo abierto más código de entrenamiento, inferencia y evaluación, bibliotecas, datos de evaluación
- Ciencia abierta: herramientas abiertas más artículo, conjuntos de datos, archivos de registro, parámetros de modelos intermedios, pesos y metadatos

Para cada uno de ellos...

- Libertad de uso
- Libertad de estudio
- Libertad de modificación
- Libertad de distribución

Las «cuatro libertades» de "¿Qué es el Software Libre?"

Algunos aspectos específicos

- Acceso: ¿puedes ejecutar inferencias de la manera que quieras?
- Control sobre el modelo: ¿puedes modificar la forma en que el modelo funciona? (por ejemplo, fine-tunar)
- Control sobre tus datos: ¿puedes controlar el prompt, los resultados?
- Autonomía: ¿cuánta dependencia tienes del proveedor del modelo?
- Confianza: ¿puedes asegurarte de que el modelo

¿Por qué esto importa?

- Acceso al modelo: casos de uso, innovación, integración
- Control del modelo: casos de uso, innovación, integración
- Control de los datos: privacidad, independencia, dependencia
- Autonomía: competencia en el mercado, independencia, dependencia
- Confianza: seguridad, transparencia.

Modelo detrás de la aplicación

- Una aplicación utiliza uno o varios modelos para proporcionar algún servicio
- Puede ser una aplicación local o en la nube
- La aplicación puede ser generalista o muy específica
- El modelo puede cambiar con el tiempo

 Understanding Free Software Licenses

Sources

+ Add

🔍 Discover

Select all sources ☒

 gbiblioteca,+1-M-barahona.pdf ☒

 <https://jgbarah.github.io/presenta...> ☒

Chat



Understanding Free Software Licenses

2 sources

Los recursos proporcionados por Jesús M. González-Barahona ofrecen una **visión completa del software libre**, explicando sus fundamentos y evolución. Se detalla que el software libre se define por **cuatro libertades clave**: usarlo sin restricciones, estudiar y modificar su funcionamiento, redistribuirlo y distribuir versiones modificadas, siendo esencial el acceso al código fuente. El autor también **aborda la historia del software libre**, desde sus orígenes hasta su madurez actual, y explora las **implicaciones para los usuarios**, incluyendo ventajas como la competencia en el mercado y la independencia del productor, así como **desafíos**. Finalmente, se hace un **examen profundo de las licencias de software libre**, diferenciando entre

Start typing...

2 sources



¿Cómo las licencias de software libre definen las libertades del usuario?

¿Qué >

Studio

 Audio Overview

⋮

 Video Overview

⋮

 Mind Map

 Reports

▼

 Understanding Free Softwar...
2 sources · 127d ago

⋮

 A Concise History of Free Software
Timeline · 2 sources · 127d ago

⋮

 Add note

NotebookLM can be inaccurate; please double check its responses.

- Google NotebookLM

11



DeepL



Start free trial

Menu



Translate text

35 languages



Translate files

.pdf, .docx, .pptx



DeepL Write

AI-powered edits

Spanish (detected) ▾



English (American) ▾

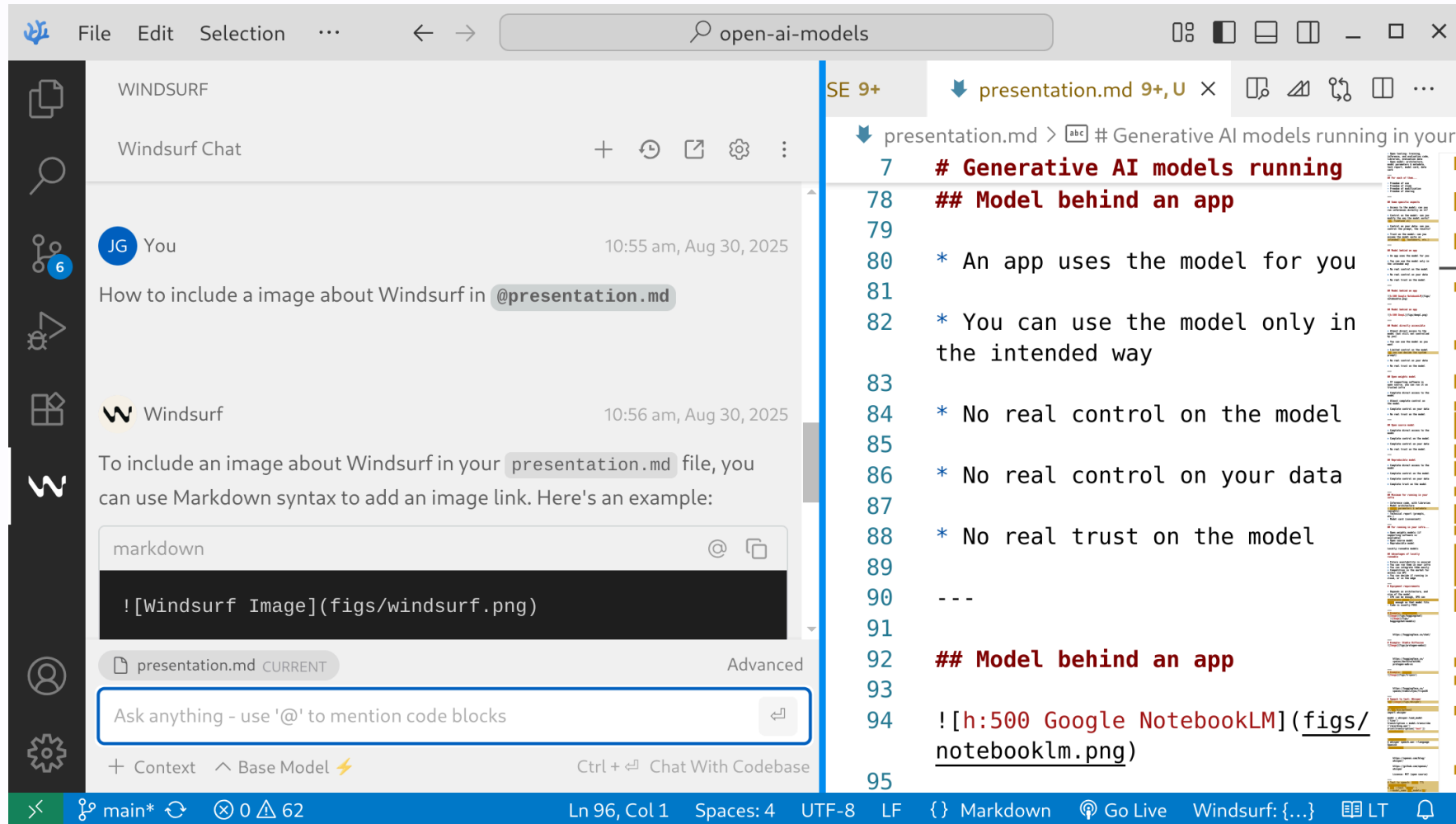
Options ▾

Un LLM al que se accede vía
una aplicación



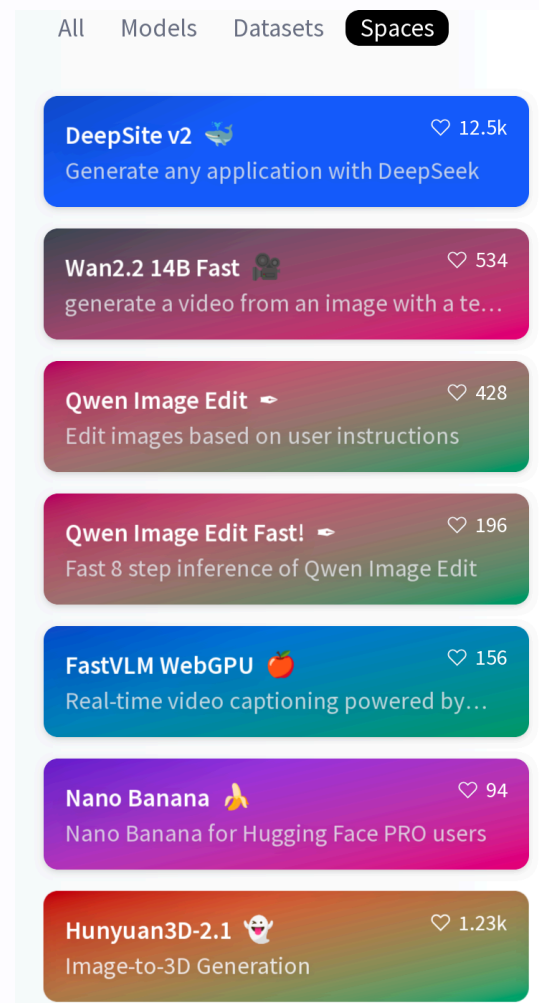
An LLM accessed via an
application

- DeepL



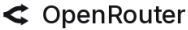
- Windsurf en Visual Studio Code

- HuggingFace Spaces



Modelo directamente accesible

- Acceso usualmente a través de una API HTTP
- La API define hasta qué punto el modelo puede ser controlado
- Bibliotecas y kits de desarrollo (SDKs) pueden estar disponibles
- Diseñados para crear aplicaciones, dependiendo de la API
- El modelo puede cambiar con el tiempo

 OpenRouter



The Unified Interface For LLMs

Better [prices](#), better [uptime](#), no subscription.



Featured Models [View Trending](#)

Gemini 2.5 Pro
by [google](#)

171.5B	2.4s	+8.82%
Tokens/wk	Latency	Weekly growth

GPT-5
by [openai](#)

69.1B	5.8s	+21.67%
Tokens/wk	Latency	Weekly growth

Claude Sonnet 4
by [anthropic](#)

624.0B	1.8s	+4.04%
Tokens/wk	Latency	Weekly growth

- OpenRouter



OpenAI GPT-OSS (20B & 120B) models now available for instant inference.
These models have built-in browser search and code execution capabilities.

[Learn about GPT-OSS >](#)

REASONING

-  GPT OSS 120B
-  GPT OSS 20B
-  DeepSeek R1 Distill Llama
-  Qwen 3 32B

FUNCTION CALLING / TOOL USE

-  GPT OSS 120B
-  GPT OSS 20B
-  Llama 4 Scout
-  Qwen 3 32B
-  Kimi K2

TEXT TO SPEECH

-  PlayAI TTS

SPEECH TO TEXT

-  Whisper Large v3
-  Whisper Large v3 Turbo

TEXT TO TEXT

-  GPT OSS 120B
-  GPT OSS 20B
-  Kimi K2
-  Llama 4 Scout
-  Llama 3.3 70B

VISION

-  Llama 4 Scout
-  Llama 4 Maverick

MULTILINGUAL

-  GPT OSS 120B
-  GPT OSS 20B
-  Kimi K2
-  Llama 4 Scout
-  Llama 3.3 70B
-  Whisper Large v3

SAFETY / CONTENT MODERATION

-  Llama Guard

• Groq

Modelo con pesos disponibles

- Los pesos están disponibles
- Normalmente, el software para inferencias está disponible / OSS
- Puede ejecutarse en infraestructura de confianza
- Fine-tuning, etc. suele ser posible
- La redistribución, modificación y uso pueden estar sujetos a condiciones o estar prohibidos

En algunos casos, se le llama «modelos de pesos abiertos»

Ejemplos de modelos con pesos disponibles

- Kimi K2
 - Licencia MIT modificada
- LLaMa3 models (abril 2025)
 - Licencia Comunitaria de Meta
 - La licencia de LLaMa de Meta sigue sin ser Open Source

Ejemplos de modelos con pesos disponibles

- **Gema3**
 - Términos de Uso de Gemma
- **Mistral Small 4** (marzo 2026)
 - Licencia Apache 2.0

Ejemplos de modelos con pesos disponibles

- **Tülu3**
 - Licencia Comunitaria de LLaMa
 - Fine-tuned a partir de LLaMa3.1
 - Todos los detalles y datos del fine-tune están disponibles
- **Cohere Command A** (agosto 2025)
 - Creative Commons Attribution-NonCommercial 4.0 International y Términos de Uso de Cohere

Modelo de peso abierto

- Permite usar, redistribuir y crear obras derivadas
- No se imponen condiciones de uso
- Obras derivadas: fine-tuning, integración, etc.
- No requiere información sobre el modelo, su entrenamiento, etc. (no hay libertad de estudio)

Definición de Open Weight

Open-Weight AI Models: What They Are, and Why
OpenAI's Next Move Matters

Ejemplo de modelo de peso abierto

- **Granite Code** (noviembre 2024)
 - Licencia Apache 2.0
 - Pocos datos sobre el modelo, su entrenamiento, etc.

Modelo de código abierto

- Permite usar, redistribuir y crear obras derivadas
 - No se imponen condiciones de uso
 - Obras derivadas: fine-tuning, integración, etc.
 - Software de código abierto para entrenamiento, inferencia
 - Descripción detallada del entrenamiento, sin necesidad de disponer del dataset de entrenamiento
- Definición de Open Source AI

¿Qué son los Open Weights?

Ejemplos de modelos de código abierto

- **DeepSeek** (septiembre 2025)
 - Licencia MIT
- **GPT-OSS** (agosto 2025)
 - Licencia Apache 2.0
- **Qwen3.5** (febrero 2026)
 - Licencia Apache 2.0

Modelo reproducible (libre)

- Permite usar, redistribuir y crear obras derivadas
- No se imponen condiciones de uso
- Obras derivadas: fine-tuning, integración, etc.
- Toda la información del modelo
- Requiere la disponibilidad del dataset de entrenamiento

Ejemplos de modelos reproducibles (libres)

- Olmo3, informe técnico (diciembre 2025)
 - Licencia Apache 2.0
- MAP-Neo (abril 2025)
 - Licencia Apache 2.0

Ejemplos de modelos reproducibles (libres)

- LLM360 K2, informe técnico (julio 2024)
 - Otros modelos LLM360
- Apertus (septiembre 2025)
 - Licencia Apache 2.0
- Alia (2026-02)
 - Apache 2.0 License

Behind-app model

Directly accessible model

Available weights model

Open weight model

Open source model

Reproducible (libre) model

	Acceso	Control del modelo	Control de datos	Autonomía	Confianza
Modelo detrás de la aplicación	Definido por la app	Ninguno	Ninguno	Ninguno	Ninguno
Modelo directamente accesible	Restricciones de la API	Restricciones de la API	Ninguno	Ninguno	Ninguno
Modelo con pesos disponibles	Con condiciones	Con condiciones	Completo	Con condiciones	Ninguno
Peso abierto	Usar como quieras	Control profundo	Completo	Estudio restringido	Ninguno
Código abierto	Usar como quieras	Control profundo	Completo	Estudio detallado restringido	Parcial
Reproducibile	Usar como quieras	Control profundo	Completo	Completo	Completo

Problemas abiertos

- Relación compleja entre datos, recetas, arquitectura, pesos, software...
 - Falta de definiciones exactas para varias categorías
 - Difícil determinar en qué categoría se encuentra un modelo
- Adaptaciones de definiciones para fine-tunes y evoluciones de modelos?
- ¿Cómo asegurar que los anuncios sean verdaderos?

Modelos auto-alojables

Mínimo para ejecutar en tu infraestructura

- Código de inferencia, con bibliotecas
- Parámetros y metadatos del modelo (pesos)
- Informe técnico (prompts, etc.)
- Tarjeta del modelo (conveniente)

Al menos, «modelo con pesos disponibles» si el código de inferencia está disponible

Modelos auto-alojables

Modelos auto-alojables

- Si el software de apoyo está disponible
 - Modelos con pesos disponibles
 - Modelos de peso abierto
- Modelos de código abierto
- Modelos reproducibles

Behind-app model

Directly accessible model

Available weights model

Open weight model

Open source model

Reproducible (libre) model

The screenshot shows the Hugging Face website interface. At the top, there's a navigation bar with the Hugging Face logo, a search bar, and links for Models, Datasets, Spaces, Docs, and Pricing. Below the navigation bar, the left sidebar contains tabs for Main, Tasks, Libraries, Languages, Licenses, and Other. The 'Tasks' section lists various tasks like Text Generation, Image-Text-to-Text, Image-to-Image, Text-to-Image, Text-to-Video, and Text-to-Speech. The 'Parameters' section features a slider for model size, ranging from < 1B to > 500B. The 'Libraries' section lists various frameworks like PyTorch, TensorFlow, JAX, Transformers, Diffusers, Safetensors, ONNX, GGUF, Transformers.js, MLX, and Keras. The main content area displays a list of models, including microsoft/VibeVoice-1.5B, xai-org/grok-2, openbmb/MiniCPM-V-4_5, Qwen/Qwen-Image-Edit, deepseek-ai/DeepSeek-V3.1, and Wan-AI/Wan2.2-S2V-14B. Each model entry shows its name, task, size, update time, and download/like statistics.

- HuggingFace Models

Ventajas de los modelos auto-alojables

- Disponibilidad futura garantizada
- Ejecutarlos en tu propia infraestructura
- Integrarlos fácilmente
- Competencia en el mercado de acceso mediante API
- Tú decides si ejecutarlos en la nube o en el borde

Desventajas de los modelos auto-alojables

- Los modelos disponibles suelen ser menos avanzados que el estado del arte
 - (posiblemente un retraso de 6-12 meses)
- La mejora y el soporte no siempre ocurren

Ventajas de auto-alojar

- Control total de todo
- Para muchos tipos de tareas, basta con lo disponible
- Más fácil para reproducibilidad

Desventajas de auto-alojar

- Inversión en infraestructura
 - Limitaciones del hardware disponible
 - El mantenimiento recae en ti
 - Mantenerse al día puede ser un dolor
 - Decisiones sobre el mejor modelo
 - Decisiones sobre el mejor software
 - Decisiones sobre el mejor hardware
- ¡Se requieren habilidades técnicas!

Requisitos de equipamiento

- Dependen de la arquitectura y el tamaño del modelo
- La CPU puede ser suficiente, la GPU acelera mucho
 - Opciones en la nube para GPUs
- Memoria RAM suficiente para que el modelo quepa
- El código suele ser OSS

También puedes desplegar en un host basado en la nube

Aspectos económicos

- Requisitos de hardware
 - Período de retorno de inversión
- Tamaño y características de la carga de trabajo
- Fuentes de complejidad:
 - Múltiples usuarios, múltiples modelos, etc.
- Costos de mantenimiento
- Consumo de energía

Tienes que hacer los cálculos

Ejecución local vía API

- Muchos proveedores diferentes con APIs similares
 - La mayoría usan la API de OpenAI
 - Muchos ofrecen varios modelos
 - Ejemplos: OpenRouter, Groq
- Frameworks proporcionan back-ends para varios proveedores
 - Ejemplos: LiteLLM, LangChain
- Fácil probar varios modelos
- Dependiendo de la carga, puede ser más barato

Cuantización

- Utilizada para reducir los requisitos de hardware
- De float32 a fp16, a int8, o menos
- Normalmente, post-entrenamiento
- Varios formatos:
 - GPTQ, suele usar archivos `safetensors`
 - GGUF, popular en el ecosistema llama.cpp

Guía de HuggingFace sobre Cuantización

GGUF: Estructura y Uso

Fine-tuning

- Adaptar un modelo a una tarea específica con un dataset (usualmente más pequeño) especializado
- Punto de partida: pesos de un modelo dado
- Ajustar los pesos para que se ajusten al dataset especializado
- Adapter: nuevas capas de pesos añadidas a un modelo para fine-tuning

¿Qué es fine-tuning?

HuggingFace: Cuantizaciones y fine-tunes

📁 Model tree for Qwen/Qwen3-8B

Base model [Qwen/Qwen3-8B-Base](#)

📁 Finetuned ([154](#)) [this model](#)

Adapters [123 models](#)

Finetunes [230 models](#)

Merges [19 models](#)

Quantizations [144 models](#)

Civit.AI: Imágenes y videos

- Fine-tuning
- Cuantización
- Imágenes y videos

<https://civitai.com/>

Motores de inferencia

- llama.cpp
 - Proporciona una UI basada en web y una API HTTP* vLLM
 - Proporciona API HTTP

Frameworks para LLMs

- **LangChain**: encadenar operaciones de grandes modelos de lenguaje en flujos de trabajo sofisticados, generalmente para crear herramientas de agente
- **LiteLLM**: conjunto ágil diseñado para eficiencia y simplicidad

Ambos pueden usar modelos locales, o modelos a través de API HTTP

Frentes de chat / asistente

- [Ollama](#), basado en llama.cpp
- [Oobabooga WebUI](#)
- [Open WebUI](#): «casi» OSS
- [LibreChat](#): online, código fuente
- [Jan](#): asistente local

La mayoría de ellos también ofrecen una API HTTP

Ollama: cómo ejecutarlo

```
curl -fsSL https://ollama.com/install.sh | sh  
ollama serve
```

```
ollama run gemma3:1b
```

```
curl http://localhost:11434/api/generate -d '{  
  "model": "gemma3:1b",  
  "prompt": "Why is the sky blue?"  
}'
```

Usar Ollama para hospedar un LLM en equipos solo con CPU para habilitar un chatbot y API local

Open WebUI: cómo ejecutarlo

```
uv venv --python 3.11  
uv pip install open-webui  
uv run open-webui serve
```

Abre <http://localhost:8080>

gemma3:1b ▾ +



Hi, WebUI, how are you

OI gemma3:1b

Hello there! As a large language model, I don't "feel" in the same way humans do, but I'm functioning perfectly and ready to help you.

I'm doing well, thank you for asking! 😊

How about you? How are *you* doing today? Is there anything you'd like to chat about or any task you'd like me to assist with?

Send a Message



 Code Interpreter

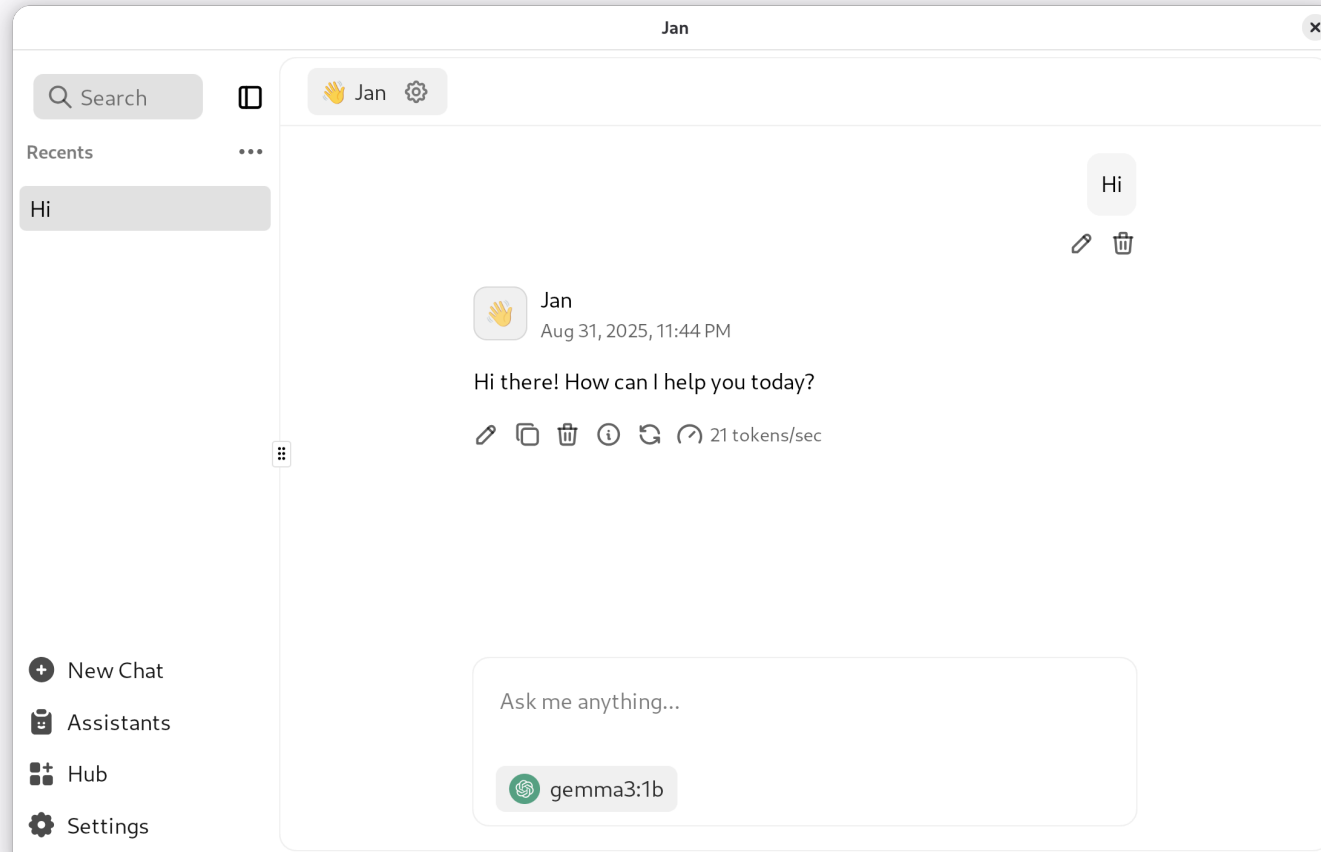


Jan: cómo ejecutarlo

- Descargar el paquete Debian (o el correspondiente a tu SO)

```
sudo pkg -i Jan_0.6.9_amd64.deb  
Jan
```

- Ajustes > Proveedores de modelo > OpenAI
- API Key «ollama»
- Base URL: <http://localhost:11434/v1>
- Modelos: añade uno nuevo («+»), «gemma3:1b»
- Selecciona el modelo en el chat



Otros modelos auto-alojables generativos

Generar imágenes

- [Qwen-Image](#), Apache 2.0 (agosto 2025)
- [HiDream-I1](#), MIT License (julio 2025)
- [FLUX.1Kontext\[dev\]](#), [modelos](#), Licencia No Comercial de Flux (agosto 2025)
- [Stable Diffusion](#), [modelos](#), Licencia Comunitaria de StabilityAI (enero 2025)

[Guía para modelos de generación de imágenes de código abierto](#)

[Arena de texto-a-imagen](#)

Generar video

- [Wan 2.2](#), [repo](#), Apache 2.0 (agosto 2025)
- [Hunyuan Video](#), [repo](#), Licencia Comunitaria de Hunyuan de Tencent
- [LTX Video](#), [repo](#), Apache 2.0* [Stable Video Diffusion](#), licencia propietaria, gratuito para algunos usos

[Arena de texto-a-imagen y video](#)

Text-to-video e imagen (apps & fine-tunes)

- [ComfyUI](#), [repo](#): interfaz y UI para varios modelos self-hostables de texto-a-imagen y texto-a-video
- [Wan2GP](#): UI y front-end para varios modelos self-hostables de texto-a-video
- [CivitAI](#): modelos y fine-tunes

Texto a texto (transcripción)

Whisper, MIT License

```
uv venv
uv pip install openai-whisper
uv run whisper speech.wav --language Spanish
```

```
#!/usr/bin/python3
import whisper

model = whisper.load_model('tiny')
transcription = model.transcribe('recording.wav')
print(transcription['text'])
```

Texto a voz

- [CoquiTTS](#), [modelo](#), Licencia de modelo público de Coqui (2023-11)

```
$ tts --text "Texto" \  
  --model_name tts_models/es/mai/tacotron2-DDC \  
  --out_path speech.wav
```

- [Kokoro](#), Licencia Apache 2.0 (enero 2025)
- [Higgs Audio V2](#), Licencia Comunitaria de Higgs Audio 2 (julio 2025)

Texto a voz (2)

- [Chatterbox](#), MIT License (abril 2025)
- [MeloTTS](#) & [OpenVoice v2](#) MIT License (febrero 2024, abril 2024)
- [FishSpeech](#), CC Attribution-NonCommercial-ShareAlike (agosto 2025) (noviembre 2024)

[Explorando el mundo de modelos de texto-a-voz de código abierto](#)

Otros modelos aleatorios

- Entender y razonar sobre series temporales: [ChatTS](#)
Apache 2.0, incluye dataset de entrenamiento
(agosto 2025)
- Mundos 3D inmersivos y explorables: [HunyuanWorld](#)
- Texto a 3D: [LlamaMesh](#), Licencia Comunitaria de LLaMa (noviembre 2024)
- Modelo y herramientas abiertas para crear videos con IA: [OpenSora]
 - [repo](#)

Otras aplicaciones

- **OpenCode**: asistente, agente
 - CLI, web y app de escritorio
- **Hive**: orquestador para agentes de codificación
- **AnythingLLM**: asistente, agente
- **FastSDCPU**: generación de imágenes optimizada para CPU
- **Lemonade**: herramienta de generación de LLM, imágenes y voz
 - Funciona bien con Vulkan, reconoce mi GPU Intel

Otras aplicaciones (2)

- **OpenClaw**: sistema agente
- **Hermes agent**: sistema agente
- **Autoresearch**: agente de auto-investigación
- **Good night, have fun**: agente tipo auto-investigación, para codificación

Otras aplicaciones

- **SurfSense**: asistente integral
 - Código fuente
- **DeerFlow**: asistente integral
 - Prueba en VolcEngine
 - Código fuente
- **Hyprnote**: herramienta de toma de notas para reuniones
- **TransformerLab**: entrenar, afinar, chatear con LLM
 - Código fuente

Conjuntos de datos de entrenamiento abiertos

- Chatbot Arena Leaderboard (Cómo funciona)
 - Datasets
- Conjuntos de datos LAION
- Common Corpus: The Largest Collection of Ethical Data for LLM Pre-Training
- The Common Pile v0.1: An 8TB Dataset of Public Domain and Openly Licensed Text

Benchmarks

- Chatbot Arena Leaderboard
- LiveBench
- LLM Leaderboard
- SWE-bench
- SWE-bench-live